# Student Registration System

Full Stack Development – TASK_2

Advanced Data Retrieval and Sorting Dashboard

## 1 Introduction

The Student Registration System (Task 2) is an enhanced version of Task 1. It includes advanced data retrieval features such as sorting student records, filtering by department, and displaying count of students per department using SQL aggregation functions.

## 2 Software Requirements

- Node.js
- MySQL Server
- MySQL Workbench
- VS Code
- Web Browser
- Git (Optional)

## 3 Project Structure

```
TASK_2/

 server.js
 package.json
 .env

 public/
    style.css

 views/
     register.ejs
     students.ejs
```

## 4 Database Design

**Database Name:** studentdb
**Table Name:** students

### 4.1 SQL Code

```sql
CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    dob DATE NOT NULL,
    department VARCHAR(100) NOT NULL,
    phone VARCHAR(15) NOT NULL
);
```

Listing 1: Database Table Structure

## 5 Updated /students Route (server.js)

```js
app.get('/students', (req, res) => {
    let sort = req.query.sort || 'name';
    let order = req.query.order || 'ASC';
    let department = req.query.department;

    let sql = 'SELECT * FROM students';
    let values = [];

    if (department && department !== '') {
        sql += ' WHERE department = ?';
        values.push(department);
    }

    if (sort === 'name' || sort === 'dob') {
        sql += ` ORDER BY ${sort} ${order === 'DESC' ? 'DESC' : 'ASC'}`;
    }

    db.query(sql, values, (err, results) => {
        if (err) throw err;

        db.query('SELECT department, COUNT(*) AS count FROM students GROUP BY department',
        (err, counts) => {
            if (err) throw err;
            res.render('students', { students: results, counts: counts });
        });
    });
});
```

Listing 2: Enhanced Students Route with Filtering and Sorting

## 6 Updated students.ejs with Filter Controls

```html
<!DOCTYPE html>
<html>
<head>
    <title>Student Dashboard</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel
    ="stylesheet">
</head>
<body class="bg-light">
    <div class="container mt-5">
        <div class="card p-4 shadow">
            <h2 class="text-center mb-4">Student Dashboard</h2>

            <!-- Filter and Sort Controls -->
            <form method="GET" class="row g-3 mb-4">
                <div class="col-md-4">
```

```
                    <label class="form-label">Sort By</label>
                    <select name="sort" class="form-select">
                        <option value="name">Name</option>
                        <option value="dob">Date of Birth</option>
                    </select>
                </div>
                <div class="col-md-4">
                    <label class="form-label">Order</label>
                    <select name="order" class="form-select">
                        <option value="ASC">Ascending</option>
                        <option value="DESC">Descending</option>
                    </select>
                </div>
                <div class="col-md-4">
                    <label class="form-label">Department</label>
                    <select name="department" class="form-select">
                        <option value="">All Departments</option>
                        <option value="CSE">CSE</option>
                        <option value="ISE">ISE</option>
                        <option value="ECE">ECE</option>
                        <option value="ME">ME</option>
                    </select>
                </div>
                <div class="col-12">
                    <button type="submit" class="btn btn-primary">Apply Filters</button>
                    <a href="/students" class="btn btn-secondary">Reset</a>
                </div>
            </form>

            <!-- Students Table -->
            <table class="table table-bordered table-hover">
                <thead class="table-dark">
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Email</th>
                        <th>DOB</th>
                        <th>Department</th>
                        <th>Phone</th>
                    </tr>
                </thead>
                <tbody>
                    <% students.forEach(student => { %>
                    <tr>
                        <td><%= student.id %></td>
                        <td><%= student.name %></td>
                        <td><%= student.email %></td>
                        <td><%= student.dob.toISOString().split('T')[0] %></td>
                        <td><%= student.department %></td>
                        <td><%= student.phone %></td>
                    </tr>
                    <% }) %>
                </tbody>
            </table>

            <!-- Department-wise Count -->
            <div class="mt-4">
                <h4>Department-wise Student Count</h4>
                <table class="table table-sm table-striped">
                    <thead>
                        <tr>
                            <th>Department</th>
                            <th>Number of Students</th>
                        </tr>
                    </thead>
                    <tbody>
                        <% counts.forEach(count => { %>
                        <tr>
```

```
                            <td><%= count.department %></td>
                            <td><%= count.count %></td>
                        </tr>
                        <% }) %>
                    </tbody>
                </table>
            </div>
        </div>
    </div>
</body>
</html>
```

Listing 3: Enhanced Students View with Filtering UI

# 7 Features Implemented

- **Display All Students** – SELECT query to retrieve all records
- **Sort by Name / DOB** – ORDER BY clause with dynamic sorting
- **Filter by Department** – WHERE clause for department-specific viewing
- **Count per Department** – GROUP BY with COUNT aggregation function
- **Interactive UI** – Dropdown controls for user-friendly interaction
- **Reset Functionality** – Clear all filters with one click

# 8 SQL Queries Demonstrated

## 8.1 Basic SELECT

```sql
SELECT * FROM students;
```

## 8.2 SELECT with WHERE Filter

```sql
SELECT * FROM students WHERE department = 'CSE';
```

## 8.3 SELECT with ORDER BY

```sql
SELECT * FROM students ORDER BY name ASC;
SELECT * FROM students ORDER BY dob DESC;
```

## 8.4 SELECT with GROUP BY and COUNT

```sql
SELECT department, COUNT(*) AS count
FROM students
GROUP BY department;
```

# 9 Steps to Execute

1. Run the server using: `node server.js`
2. Open browser and navigate to: `http://localhost:3000/students`
3. Use dropdown filters for sorting and filtering:
   - Sort by Name or Date of Birth
   - Choose Ascending or Descending order
   - Filter by specific department
4. Click "Apply Filters" to see filtered and sorted results
5. Department-wise count appears below the table
6. Click "Reset" to clear all filters and view all students

# 10 Expected Output Examples

### When sorted by Name (ASC):

- Students displayed in alphabetical order
- A $\rightarrow$ Z ordering based on student names

### When filtered by Department 'CSE':

- Only Computer Science students visible
- Department count table shows all departments with their counts

### When grouped by Department:

- Summary table shows: CSE: 5, ISE: 3, ECE: 4, ME: 2
- Useful for quick statistical overview

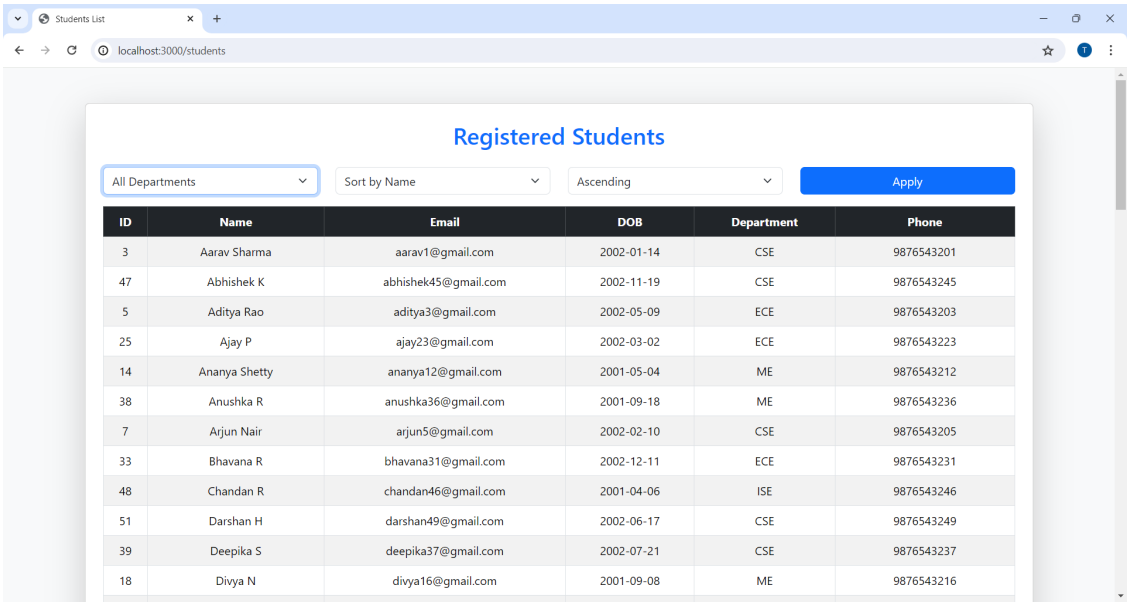


Figure 1: Selecting Values To Be Sorted

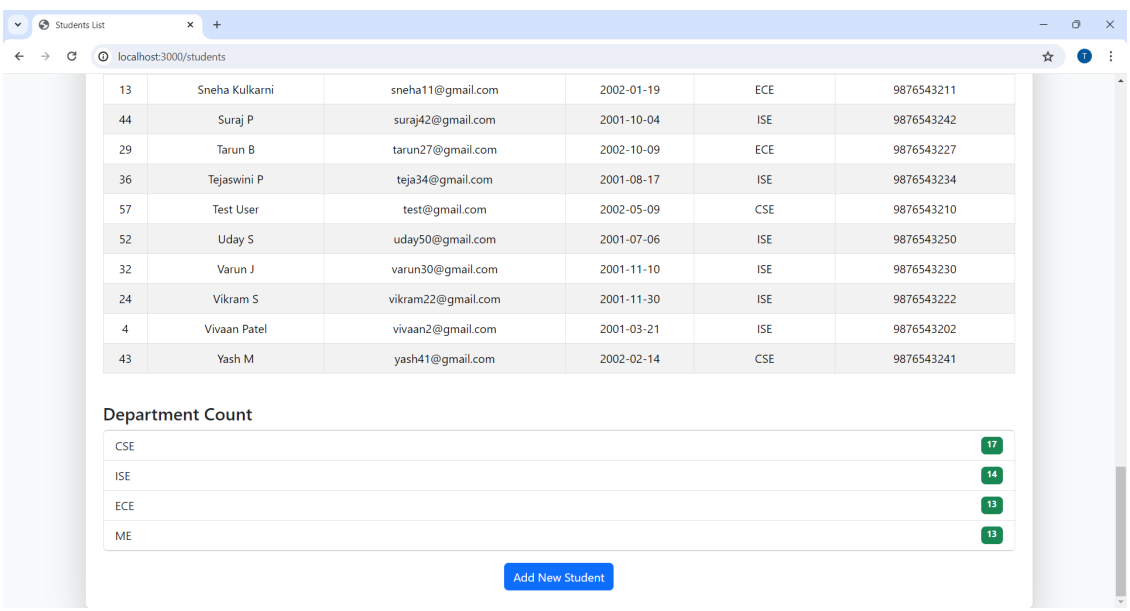


Figure 2: Department Count

# 11 Conclusion

Task 2 enhances the Student Registration System by implementing dynamic filtering, sorting, and aggregation using SQL queries. This demonstrates advanced data retrieval techniques in Full Stack Development including:

- Dynamic SQL query construction
- Multi-criteria filtering

- Data aggregation and reporting
- Interactive user interface for data manipulation

The system effectively showcases how backend database operations can be combined with frontend controls to create a powerful data dashboard.